➢ Technical Report: Smart Rock Paper Scissors System

1. Executive Summary

The **Smart Rock Paper Scissors** application is a console-based implementation of the classic hand game. Unlike standard implementations that rely solely on random number generation, this version utilizes a frequency-based heuristic to adapt to player behavior, creating a more challenging "Smart AI" experience.

2. System Architecture

The program is built using a **procedural programming** paradigm in C++. It is divided into logical modules to ensure maintainability and readability.

2.1 Core Components

- **Game Loop:** A do-while structure that manages the session state and allows for multiple rounds.
- **Input Validation:** A robust mechanism using cin.fail() to prevent program crashes from non-integer inputs.
- **Predictive Engine:** A decision-making algorithm that analyzes historical player data.
- **State Management:** Tracking variables for wins, losses, draws, and move frequency.

3. Key Features and Logic

3.1 Adaptive AI Algorithm

The standout feature of this program is the getComputerChoice function. It operates on a **Frequency Analysis** model:

1. **Warm-up Phase:** For the first three rounds, the AI generates purely random moves to gather a baseline of player data.
2. **Strategic Phase:** The AI identifies the player's most frequent move and selects the counter-move.
    - *Example:* If the player favors "Rock," the AI will prioritize "Paper."

3.2 Robust Input Handling

The system includes defensive programming to handle "dirty" user input.

- **Type Checking:** Uses cin.clear() and cin.ignore() to flush the input buffer if a user enters text instead of a number.
- **Range Checking:** Ensures the integer provided is strictly within the $1 \dots 3$ range.

4. Functionality Overview

Function	Purpose
getPlayerChoice()	Captures and validates user input.
getComputerChoice()	Implements the "Smart" logic based on move history.
determineWinner()	Evaluates the win/loss/draw condition using boolean logic.
showScoreboard()	Displays a formatted summary of the current session statistics.

5. Technical Specifications

- **Language:** C++
- **Libraries Used:** * <iostream>: Standard I/O.
    - <cstdlib>&<ctime>: Used for seeding the random number generator ($srand$).
    - <limits>: Used for clearing the input stream.
- **Logic Complexity:** The win condition is evaluated in $O(1)$ time complexity using a series of compound conditional statements.

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <limits>

using namespace std;

// Function prototypes
void showMenu();
int getPlayerChoice();
int getComputerChoice(int rockCount, int paperCount, int scissorCount);
string determineWinner(int player, int computer);
string choiceToWord(int choice);
void showScoreboard(int playerScore, int computerScore, int draws);

// ================= MAIN =================
int main()
{
    srand(time(0));

    char playAgain;

    // AI memory
    int rockCount = 0, paperCount = 0, scissorCount = 0;

    // Scoreboard
    int playerScore = 0, computerScore = 0, draws = 0;

    cout << "===== SMART ROCK PAPER SCISSORS =====\n";
```

```cpp
    do
    {
        showMenu();

        int player = getPlayerChoice();

        // Update history for AI learning
        if (player == 1) rockCount++;
        else if (player == 2) paperCount++;
        else scissorCount++;

        int computer = getComputerChoice(rockCount, paperCount, scissorCount);

        cout << "\nYou chose: " << choiceToWord(player) << endl;
        cout << "Computer chose: " << choiceToWord(computer) << endl;

        string result = determineWinner(player, computer);
        cout << "\nResult: " << result << endl;

        // Update scores
        if (result == "You Win!") playerScore++;
        else if (result == "Computer Wins!") computerScore++;
        else draws++;

        // Show scoreboard
        showScoreboard(playerScore, computerScore, draws);

        cout << "\nPlay again? (y/n): ";
        cin >> playAgain;
```

```
    } while (playAgain == 'y' || playAgain == 'Y');

    cout << "\nFinal ";
    showScoreboard(playerScore, computerScore, draws);

    cout << "\nThanks for playing!\n";
    return 0;
}

// ================= MENU =================
void showMenu()
{
    cout << "\n1. Rock\n";
    cout << "2. Paper\n";
    cout << "3. Scissors\n";
}

// ================= INPUT VALIDATION =================
int getPlayerChoice()
{
    int choice;

    while (true)
    {
        cout << "Enter your choice (1-3): ";
        cin >> choice;

        if (cin.fail())
        {
            cin.clear();
```

```
            cout << "Invalid input! Enter numbers only.\n";
            continue;
        }

        if (choice < 1 || choice > 3)
        {
            cout << "Choice must be between 1 and 3.\n";
            continue;
        }

        return choice;
    }
}

// ================= SMART AI =================
int getComputerChoice(int rockCount, int paperCount, int scissorCount)
{
    // First few rounds random
    if (rockCount + paperCount + scissorCount <= 3)
        return rand() % 3 + 1;

    // Predict most frequent player move
    if (rockCount >= paperCount && rockCount >= scissorCount)
        return 2; // Paper beats Rock

    else if (paperCount >= rockCount && paperCount >= scissorCount)
        return 3; // Scissors beats Paper

    else
        return 1; // Rock beats Scissors
```

```
// ================= WORD CONVERSION =================
string choiceToWord(int choice)
{
    if (choice == 1) return "Rock";
    if (choice == 2) return "Paper";
    return "Scissors";
}

// ================= WINNER LOGIC =================
string determineWinner(int player, int computer)
{
    if (player == computer)
        return "Draw!";

    if ((player == 1 && computer == 3) ||
        (player == 2 && computer == 1) ||
        (player == 3 && computer == 2))
        return "You Win!";

    return "Computer Wins!";
}

// ================= SCOREBOARD =================
void showScoreboard(int playerScore, int computerScore, int draws)
{
    cout << "\n===== SCOREBOARD =====\n";
    cout << "Player Wins   : " << playerScore << endl;
    cout << "Computer Wins : " << computerScore << endl;
    cout << "Draws         : " << draws << endl;
}
```

6. Potential Future Enhancements

To evolve the "Smart" aspect of the game, the following features could be implemented:

1. **Pattern Recognition:** Transition from simple frequency analysis to **Markov Chains** to predict sequences (e.g., if a player always plays "Paper" after "Rock").
2. **File I/O:** Save player history to a .txt or .dat file so the AI "remembers" the player even after the program is closed.
3. **GUI Implementation:** Migrate the console interface to a graphical library like SFML or Qt.

7. Conclusion

The provided source code is a clean, efficient, and user-friendly implementation of Rock Paper Scissors. It successfully bridges the gap between a simple beginner project and a more advanced application by incorporating data-driven decision-making for the computer opponent.